Análisis de módulos de simulación

Las cotas son de peor caso. Se consideran O(1) esperado para búsquedas en diccionarios y conjuntos. Las funciones de Warehouse que construyen o validan el grafo se tratan como costos externos cuando su costo depende de la implementación de dominio.

Variables

- N: productos; O: pedidos; I: líneas de pedido; R: racks; Ais: pasillos; P: pickers; W: waves.
- C: celdas de circulación; V y E: vértices y aristas del grafo; D: destinos de despacho candidatos; S: accesos o paradas de una ruta.
- T: tareas pendientes que un picker podría probar; B: costo de una búsqueda de camino, incluida su reconstrucción.
- M: total de elementos que mantiene el editor (racks, zonas, puertas, pasillos, pickers y productos); J: tamaño en bytes del JSON procesado.
- L_state: tamaño del estado guardado de la simulación, incluidos recorridos, tareas y registros.

1. Carga de entidades: Cargador

Idea y estructuras

cargar_warehouse convierte los diccionarios de un escenario en objetos de dominio. Inserta puertas, zonas, racks, pasillos, pickers, productos y pedidos. Los productos se ordenan por rack y nivel para agregarlos desde los niveles inferiores. Los ítems de pedido conservan referencias a los productos cargados.

Paso a paso

1. Crea el Warehouse con las dimensiones recibidas.
2. Recorre cada colección serializada y crea su entidad de dominio.
3. Ordena los productos por (rack_id, z) y los agrega al warehouse.
4. Resuelve cada ID de producto de los pedidos; si no existe, informa un error. Finalmente arma los ítems y pedidos.

Complejidad

Con N productos y I líneas de pedido, el ordenamiento cuesta O(N log N) y las inserciones cuestan O(R + zonas + puertas + Ais + P + N + O + I), suponiendo inserciones y búsquedas esperadas O(1) en los mapas del warehouse. El total es O(N log N + R + zonas + puertas + Ais + P + O + I). El espacio adicional es O(N + I) por el ordenamiento y los objetos creados; el warehouse también mantiene sus propias colecciones.

2. Generación y validación de escenarios: Scenario

Idea y estructuras

generar_escenario arma una geometría regular, ubica productos y pickers, genera pedidos y valida el resultado. Usa listas de diccionarios para racks, pasillos, productos y pedidos; conjuntos y diccionarios auxiliares controlan ubicaciones y stock. generar_vista_previa construye un layout determinista para la interfaz, sin armar el grafo ni comprobar alcanzabilidad.

Paso a paso

1. Valida dimensiones, capacidades, cantidades y modo de ubicación.
2. Calcula filas y columnas, crea racks, pasillos, puertas y zonas de despacho.
3. Enumera posiciones de rack para productos, baraja o prioriza las posiciones y asigna niveles.
4. Genera pedidos recorriendo productos con stock y distribuye unidades sin superar el disponible.
5. Para ubicar pickers carga el warehouse, obtiene sus celdas transitables y elige posiciones distintas.
6. validar_escenario carga las entidades, construye el grafo y verifica rutas. Si hay productos inaccesibles, guarda sus IDs y el mensaje en el escenario para que la simulación pueda continuar excluyéndolos.

Complejidad

Sea U la cantidad total de ubicaciones posibles de producto y I la cantidad de líneas creadas. Enumerar y barajar ubicaciones cuesta O(U) para los modos aleatorio/agrupado y hasta O(U log U) para cerca_despacho; elegir productos cuesta O(N). Generar pedidos recorre los N productos por pedido, por lo que cuesta O(O·N + I) en el peor caso. La creación del layout y de pickers cuesta O(R + Ais + C + P), sin contar el armado del grafo. La validación cuesta O(N log N + R + Ais + P + O + I + T_grafo + N·D·B): la última parte representa, como cota conservadora, las búsquedas de ida y vuelta de cada producto hacia destinos de despacho. T_grafo es el costo de construcción y validaciones internas del warehouse. El espacio es O(U + N + O + I + V + E) incluyendo datos, entidades y grafos.

3. Modelo de edición: ModeloEscenario

Idea y estructuras

El modelo mantiene listas de elementos serializados, un historial acotado a 100 instantáneas y contadores de IDs. Las reglas geométricas compartidas se resuelven con funciones de domain.Validation. No hay un índice espacial: las búsquedas de solapamiento, elemento bajo el cursor, rack o picker recorren las listas correspondientes.

Paso a paso

1. error valida dimensiones y límites, y compara el rectángulo nuevo con los elementos que bloquean a la herramienta.
2. agregar, mover_picker, agregar_producto, borrar y establecer_peso_zona verifican la operación, guardan una instantánea y actualizan el modelo y su versión.
3. deshacer restaura la última instantánea.
4. validar comprueba requisitos del editor y delega la validación estructural y de alcanzabilidad a validar_escenario.
5. a_diccionario copia y transforma las listas al formato de escenario JSON.

Complejidad

Para M elementos, una comprobación de colocación cuesta O(M) por los recorridos lineales de las listas bloqueantes. rack_en, picker_en, siguiente_z y borrar cuestan O(R), O(P), O(N) y O(M + N) respectivamente. Guardar una instantánea usa O(M) tiempo y espacio por deepcopy; como el historial conserva hasta 100 estados, su espacio es O(100·M), es decir O(M) asintótico con ese límite fijo. Exportar el escenario cuesta O(M + N) por las copias. validar agrega el costo de validar_escenario descrito en la sección anterior.

4. Navegación: Navegacion y GrafoFiltrado

Idea y estructuras

Navegacion reutiliza los grafos completo y reducido del warehouse. Precalcula vecinos, sentido y eje de pasillos, accesos a racks, celdas de despacho e intersecciones. GrafoFiltrado presenta una vista del grafo que omite vecinos ocupados sin copiarlo completo.

Paso a paso

1. Valida el algoritmo de búsqueda y asegura que exista el grafo reducido.
2. Recorre pasillos y racks para construir metadatos geométricos y accesos.
3. camino normaliza origen, destinos y ocupaciones; filtra destinos no válidos.
4. Si hay celdas ocupadas, consulta el grafo a través de GrafoFiltrado; para cada destino válido ejecuta Dijkstra o A* y conserva la ruta de menor costo.
5. verificar_alcanzables comprueba rutas dirigidas entre despacho, productos y posiciones iniciales de pickers.

Complejidad

La inicialización, una vez disponibles los grafos, cuesta O(V + E + C_racks + C_pasillos + C_despacho), donde los términos de celdas cubren las geometrías recorridas para accesos y zonas. El espacio de los índices auxiliares es O(V + C_racks + C_pasillos + C_despacho); los grafos reutilizados no se duplican.

Para d destinos válidos, camino hace hasta d búsquedas: O(d·B + d + q), donde q es el número de celdas ocupadas usado para construir el conjunto. La vista filtrada no copia aristas: consultar una lista de vecinos filtra esa lista al vuelo, por lo que el costo de expansión forma parte de B. La verificación de accesibilidad cuesta hasta O(N·a·D·B) con a accesos candidatos por producto y D celdas de despacho; a suele estar limitado por la geometría de los racks.

5. Estado individual: EstadoPicker

Idea y estructuras

EstadoPicker envuelve la entidad de dominio y conserva el estado operativo: posición anterior, ruta, tarea objetivo, tareas pendientes y recogidas, destinos, recorrido histórico (trail), wave activa y contadores de espera/patrulla. La entidad original sigue siendo la fuente de posición, ruta y capacidad.

Paso a paso

1. Inicializa listas, conjuntos, estado disponible y contadores.
2. asignar_viaje guarda los metadatos del trabajo actual.
3. a_dict crea una instantánea serializable de estado, ruta, tareas y viaje.

4. restaurar reconstruye coordenadas, colecciones y el diccionario de tareas, y repone la tarea objetivo.

Complejidad

La construcción y asignar_viaje cuestan O(1) sin contar las tareas ya entregadas como argumento. a_dict y restaurar cuestan O(R_p + T_p + H_p), donde R_p es la ruta, T_p las tareas y H_p el historial de recogidas/pasos copiado. Las estructuras restauradas ocupan O(R_p + T_p + H_p).

6. Conflictos de movimiento: GestorColisionesV

Idea y estructuras

El gestor coordina a los P pickers y guarda en reservas las celdas que ya fueron tomadas durante el tick actual. La simulación aporta callbacks para replanificar, registrar avances y finalizar una entrega.

Paso a paso

1. Al iniciar un tick, limpiar_reservas descarta las reservas previas.
2. orden_movimiento ordena pickers con trabajo antes que los que deambulan, desempata por ID.
3. mover revisa la siguiente celda; si está ocupada, el picker espera o cede el paso si deambula.
4. Si la celda es adyacente y está libre, la reserva, actualiza posición y ruta, y notifica el avance.
5. Si un picker que regresa alcanza despacho, llama al callback de entrega.

Complejidad

limpiar_reservas reemplaza el diccionario en O(1) tiempo; el mapa contiene hasta P celdas. orden_movimiento cuesta O(P log P). ocupadas crea un conjunto recorriendo pickers y reservas, O(P). mover cuesta O(P + R_p) por comprobar ocupación y quitar el primer punto de la ruta (list.pop(0) desplaza los restantes); los callbacks de replanificación o entrega agregan su propio costo. En un tick, el costo de coordinación y ordenamiento es $O(P^2 + P \log P + \Sigma R_p)$ si se llama a ocupadas una vez por picker.

7. Ejecución: Simulacion

Idea y estructuras

Simulacion coordina warehouse, navegación, asignador, pickers, colisiones, waves, stock, logs y métricas. Cada picker se mueve una celda por tick. El trabajo termina cuando no quedan tareas asignables ni viajes en curso; los pickers libres pueden seguir deambulando sin impedir que la simulación se marque terminada.

Paso a paso

1. reiniciar carga el warehouse, arma o reutiliza el grafo, prepara navegación y pickers, filtra productos inaccesibles, carga las waves, calcula puntos de patrulla y pide trabajo al asignador.
2. paso incrementa el tick, limpia reservas, abandona la vista previa al primer paso y procesa pickers por prioridad.
3. _avanzar elige tarea, recoge unidades, planea el regreso y solicita al gestor mover al picker.

4. Si hay bloqueos prolongados, _resolver_deadlocks prioriza por tiempo de espera e intenta una ruta alternativa.
5. Al entregar, libera el picker y vuelve a solicitar trabajo al asignador.
6. ejecutar repite ticks hasta que termina o llega al límite. estado crea una instantánea para interfaz/guardado; restaurar reconstruye asignaciones y vuelve a descontar el stock recogido.

Complejidad de un tick

No hay una cota útil que dependa solo del número de pickers: el movimiento puede disparar búsquedas de camino y el asignador tiene su propio costo (analizado en el documento de asignación). Sea T el máximo de tareas pendientes que se prueban para un picker, a el máximo de accesos consultados por tarea y D la cantidad de destinos de despacho. Sin cachés, la selección de objetivo cuesta hasta O(T·a·B) por picker; planear el regreso o replanificar cuesta hasta O(D·B). La fase de deadlock puede intentar replanificar varios pickers, hasta O(P·D·B) en el peor caso. Sumando la coordinación de colisiones:

$T_{paso} = O(P \cdot T \cdot a \cdot B + P \cdot D \cdot B + P^2 + P \log P + \Sigma R_p + T_{asignación})$.

$T_{asignación}$ es la suma del trabajo del asignador invocado por entregas/asignaciones de ese tick. Deambular puede agregar búsquedas de ruta hasta un punto de patrulla. Las cachés del planificador pueden reducir el tiempo observado, pero esta cota no presupone aciertos.

ejecutar(K) cuesta la suma de hasta K pasos, es decir $O(\Sigma T_{paso})$ hasta terminar o alcanzar max_ticks; la cantidad real de ticks depende de distancias, esperas y bloqueos. reiniciar incluye carga/validación de warehouse, construcción del grafo, inicialización de waves y asignación inicial, así que no se reduce a O(P). estado cuesta y ocupa $O(L_{state})$ porque copia listas, tareas, waves, stock y registros para devolver una instantánea.

Espacio

El estado persistente usa O(V + E + N + P + W + rutas + tareas + logs) más las cachés del planificador y asignador. Las rutas e historiales (trail) crecen con la ejecución; el log conserva como máximo 200 mensajes. Las cachés no tienen una cota fija en estos módulos.

8. Coordinación de interfaz: Sesion

Idea y estructuras

Sesion ofrece comandos de iniciar, pausar, continuar, avanzar un paso, cambiar estrategia, cargar/generar escenarios y guardar o restaurar partidas. Mantiene una instantánea de la simulación y conjuntos de productos pendientes/recogidos para la vista. La lógica temporal de la simulación permanece en Simulacion.

Complejidad

Crear o reemplazar una sesión delega a validación, construcción de Simulacion y asignación inicial. _armar_pilas recorre N productos, y _refrescar_estado obtiene la instantánea $O(L_{state})$ y recorre pendientes/recogidos. posiciones cuesta O(P). Los comandos simples de pausa/continuar/iniciar cuestan O(1) salvo que iniciar deba reiniciar; dar_paso cuesta $O(T_{paso} + L_{state})$ por el avance y la actualización de estado. Cambiar la configuración del recorrido antes del primer tick reconstruye la simulación y vuelve a pagar el costo de inicialización.

9. **Persistencia:** Persistence

Idea y estructuras

El módulo diferencia escenarios de partidas guardadas, comprueba versión y campos requeridos, valida el escenario asociado y serializa JSON en UTF-8. Para guardar escribe primero un archivo temporal y luego reemplaza el destino, de modo que el archivo anterior no se sustituya hasta completar la escritura.

Paso a paso

1. guardar_escenario valida el escenario y lo serializa.
2. guardar_partida toma simulacion.estado(), agrega datos del escenario y versión, y escribe el JSON.
3. cargar_escenario y cargar_partida leen el JSON, detectan si corresponde al tipo solicitado y validan su estructura.
4. La restauración completa del estado queda a cargo de Simulacion.restaurar, invocada por Sesion.

Complejidad

La lectura y escritura JSON cuesta O(J) tiempo y O(J) espacio transitorio por materializar contenido/serialización. La validación estructural del estado cuesta O(P + W + tareas guardadas); la validación del escenario agrega el costo de validar_escenario. El reemplazo del archivo temporal es O(1) en cantidad de datos desde el punto de vista algorítmico del programa. La partida completa ocupa O(J) en disco.

Resumen

Módulo	**Costo dominante**	**Espacio auxiliar**
Cargador	O(N log N + R + Ais + P + O + I)	O(N + I)
Scenario	O(U log U + O·N + T_grafo + N·D·B) como cota general	O(U + N + O + I + V + E)
ModeloEscenario	búsquedas lineales O(M); instantánea O(M)	historial O(M) con límite fijo de 100
Navegacion.camino	O(d·B + q)	O(d + q + V) por búsqueda
EstadoPicker	O(R_p + T_p + H_p) al serializar/restaurar	O(R_p + T_p + H_p)
GestorColisionesV	hasta O(P² + P log P + ΣR_p) por tick	O(P) para ocupación/reservas
Simulacion.paso	búsquedas de ruta + colisiones + T_asignación	estado del warehouse y de la ejecución
Sesion	O(T_paso + L_state) por paso visible	O(L_state + N) en sus índices/copia
Persistence	O(J) más validaciones	O(J) transitorio

Estas cotas explican el costo máximo, no el tiempo típico. La disposición del grafo, los destinos candidatos, la congestión, las cachés y las operaciones esperadas de diccionarios afectan el tiempo observado.